

Epoch-Fenced Garbage Collection for CRDTs

using an Object Store as Coordination Medium

Formal Specification and Correctness Proofs

Dimitrios Bakalis

Contents

I	Protocol Specification	3
1	Preliminaries	3
1.1	Identifiers and Dots	3
1.2	Version Vectors	3
1.3	Dot Version Vector	3
2	System Model	3
2.1	Abstract CRDT	3
2.2	Axioms	4
2.3	Object Store	4
2.4	Causal Stability	5
2.5	Replica State	5
3	Invariants	5
4	Protocol Procedures	6
4.1	Normal Operation	6
4.2	Computing the Stable Timestamp	6
4.3	GC Initiation	8
4.4	Joining a New Replica	9
4.5	Observing an Epoch Change	10
4.6	Epoch Entry Cleanup	10
II	Correctness Proofs	11
5	Lemma 1: LocalWrite Preserves Clock Soundness	11
6	Lemma 2: MergeDelta Preserves Clock Soundness	11
7	Lemma 3: No Future Dot Below Threshold	11
8	Theorem 1: Unique GC Initiator	12
9	Theorem 2: Initiator Currency	12
10	Theorem 3: Closed-World Safety	12

11 Theorem 4: Causal Floor Establishment	12
12 Theorem 5: GC Safety	13
13 Theorem 6: ObserveEpochChange Correctness	13
14 Theorem 7: Bootstrap Correctness	13
15 Theorem 8: Inductive Clock Soundness	14
16 Theorem 9: Strong Eventual Consistency	14
17 Theorem 10: Strict Threshold Monotonicity	14
18 Proof Dependency Table	15

Part I

Protocol Specification

1 Preliminaries

1.1 Identifiers and Dots

Definition 1.1 (Replica Identifier). Let $\mathcal{R}$ be a countably infinite set of *replica identifiers*. Each $r \in \mathcal{R}$ is a unique process. The set of *live* replicas at any moment is a finite subset $\mathcal{M} \subseteq \mathcal{R}$.

Definition 1.2 (Dot). A *dot* is a pair $d = (i, n)$ where $i \in \mathcal{R}$ is a replica identifier and $n \in \mathbb{N}^+$ is a strictly positive counter. The set of all dots is $\mathcal{D} = \mathcal{R} \times \mathbb{N}^+$. We write $d.\text{replicaId}$ and $d.\text{counter}$ for the projections.

1.2 Version Vectors

Definition 1.3 (Version Vector). A *version vector* is a partial map $V : \mathcal{R} \rightarrow \mathbb{N}$, defaulting to 0 for absent entries. The set of all version vectors is $\mathbb{V}$.

Definition 1.4 (Dot Domination). Dot $d = (i, n)$ is *dominated* by $V \in \mathbb{V}$, written $d \leq V$, iff $V(i) \geq n$.

Definition 1.5 (Version Vector Order, Join and Meet). $V_1 \leq V_2$ iff $\forall i : V_1(i) \leq V_2(i)$. The *join* (least upper bound) and *meet* (greatest lower bound) are:

$$(V_1 \sqcup V_2)(i) = \max(V_1(i), V_2(i)) \quad (V_1 \sqcap V_2)(i) = \min(V_1(i), V_2(i))$$

Both operations extend pointwise to finite sets: $\bigsqcup_{r \in \mathcal{M}} V_r$ and $\bigsqcap_{r \in \mathcal{M}} V_r$.

1.3 Dot Version Vector

Each replica r maintains a *dot version vector* $\mathcal{V}_r = (\text{dot}_r, CC_r)$ where:

- $\text{dot}_r = (r, n_r) \in \mathcal{D}$ is the latest dot generated by r (initially $n_r = 0$).
- $CC_r : \mathcal{R} \setminus \{r\} \rightarrow \mathbb{N}$ records the highest counter seen from each other replica.

Definition 1.6 (Flattened Version Vector). The *flattened* version vector $\widehat{\mathcal{V}}_r : \mathcal{R} \rightarrow \mathbb{N}$ is:

$$\widehat{\mathcal{V}}_r(i) = \begin{cases} n_r & \text{if } i = r \\ CC_r(i) & \text{otherwise} \end{cases}$$

All clock comparisons and domination checks use $\widehat{\mathcal{V}}_r$.

Definition 1.7 (Dot Version Vector Merge). $\widehat{\mathcal{V}}_r \sqcup \widehat{\mathcal{V}}_s = \lambda i. \max(\widehat{\mathcal{V}}_r(i), \widehat{\mathcal{V}}_s(i))$. After merging, r updates CC_r to reflect the join on all entries except its own dot, which remains dot_r .

2 System Model

2.1 Abstract CRDT

Definition 2.1 (CRDT). A *CRDT* is a tuple $(\mathcal{S}, \sqcup, \perp, \text{dots}, q)$ where $\mathcal{S}$ is the state space; $\sqcup$ is a commutative, associative, idempotent join; $\perp$ is the empty state; $\text{dots} : \mathcal{S} \rightarrow 2^{\mathcal{D}}$ maps states to their dot sets; and $q : \mathcal{S} \rightarrow \mathcal{V}$ is the *query function*, mapping states to observable results in a result domain $\mathcal{V}$ [Shapiro et al., 2011a, Almeida et al., 2016]. The induced order is $s_1 \sqsubseteq s_2 \iff s_1 \sqcup s_2 = s_2$.

Definition 2.2 (Observable Equivalence). Two states $s_1, s_2 \in \mathcal{S}$ are *observably equivalent*, written $s_1 \simeq s_2$, iff $q(s_1) = q(s_2)$, i.e. they return the same result under the query function. Observable equivalence is strictly weaker than state equality: two states may differ in their internal dot sets yet produce identical query results.

Definition 2.3 (GC-Eligible Dots). A dot $d \in \text{dots}(s)$ is *GC-eligible* in state s with threshold V , written $d \in \text{elig}(s, V)$, iff $d \leq V$ and removing d preserves $q(s)$, i.e. the resulting state is observably equivalent to s . The CRDT specifies its own eligibility predicate.

Remark 2.1 (Structural Realisation of GC Eligibility). Definition 2.3 states GC eligibility semantically: a dot is eligible if its removal does not change the query result. In practice, CRDTs realise this predicate structurally. In an OR-Set [Bieniusa et al., 2012], a dot (r, n) is eligible iff the element it tagged has subsequently been removed and no concurrent re-add carries a newer dot for the same element. In a Counter CRDT, all dominated dots are eligible since the query sums only the current maximum per replica. The semantic definition is general enough to encompass all such cases; the CRDT implementation is responsible for providing an efficiently computable elig predicate that satisfies it.

Definition 2.4 (GC Function). $\text{gc}(s, V)$ removes GC-eligible dots dominated by V :

$$\text{dots}(\text{gc}(s, V)) \subseteq \text{dots}(s) \setminus \text{elig}(s, V) \quad \text{and} \quad \text{gc}(s, V) \simeq s$$

2.2 Axioms

The protocol is correct for any CRDT satisfying the following axioms.

Axiom 2.1 (Join is LUB). $\sqcup$ is the least upper bound in $(\mathcal{S}, \sqsubseteq)$: commutative, associative, and idempotent.

Axiom 2.2 (Dot Monotonicity). $\text{dots}(s_1) \subseteq \text{dots}(s_1 \sqcup s_2)$.

Axiom 2.3 (GC Eligible Dot Removal). $\text{dots}(\text{gc}(s, V)) \subseteq \text{dots}(s) \setminus \text{elig}(s, V)$. GC only removes dots; it never introduces new ones.

Remark 2.2 (GC Correctness via Causal Stability). The correctness of GC in this protocol follows directly from causal stability at the system level: V_{stable} is defined as the meet of all live replica clocks (Definition 2.7), so every dot $d \leq V_{\text{stable}}$ has been seen by every live replica by construction. No live replica will ever generate a dot below V_{stable} after it is committed (Lemma 7.1), making removal of such dots structurally safe without any additional per-replica assumption.

2.3 Object Store

Definition 2.5 (Object Store). The *object store* $\mathcal{O}$ is a shared durable key-value store with *linearizable* reads and writes, supporting:

- **read**(k): returns the current value or $\perp$ if absent.
- **write**(k, v): unconditional write.
- **cas**($k, \perp, v$): atomic write-if-absent; returns *true* iff k was absent and the write succeeded.
- **listKeys**($prefix$): returns the set of all present keys with the given prefix.
- **delete**(k): removes k .

All operations are linearizable: each takes effect atomically at some point between its invocation and response [Herlihy and Wing, 1990].

Definition 2.6 (Object Store Keys). The protocol uses the following namespaced keys:

Key	Written by	Content
$member[r]$	replica r on join	presence signals r is live
$clock[r]$	replica r periodically	$\hat{\mathcal{V}}_r$
$gc_intent[N]$	GC initiator	epoch N ; written via CAS
$epoch_entry[N]$	GC initiator	V_{stable}^N
$bottom_state[N]$	GC initiator	$(gc(state_{r_i}, V_{stable}^N), \hat{\mathcal{V}}_{r_i})$

where r_i denotes the initiator of epoch N . The initiator's clock $\hat{\mathcal{V}}_{r_i}$ is stored alongside the GC'd state so that bootstrapping replicas can establish correct clock coverage for all dots in the snapshot (Section 4.4).

2.4 Causal Stability

Definition 2.7 (Causally Stable Timestamp). Given a live membership set $\mathcal{M}$ and their stored clocks $\{\hat{\mathcal{V}}_r\}_{r \in \mathcal{M}}$, the *causally stable timestamp* is:

$$V_{stable}(\mathcal{M}) = \bigcap_{r \in \mathcal{M}} \hat{\mathcal{V}}_r = \lambda i. \min_{r \in \mathcal{M}} \hat{\mathcal{V}}_r(i)$$

A dot d is *causally stable* iff $d \leq V_{stable}(\mathcal{M})$, i.e. every live replica has seen d .

Remark 2.3 (V_{stable} is the Unique Optimal Threshold). $V_{stable}(\mathcal{M})$ is simultaneously:

- **Safe:** every dot $d \leq V_{stable}$ has been seen by every live replica and, by Lemma 7.1, no live replica will ever generate such a dot again after V_{stable} is committed.
- **Optimal:** any $V' > V_{stable}$ would include a dot not yet seen by at least one live replica, making GC unsafe for that replica.

V_{stable} is therefore the greatest threshold that is simultaneously safe for all live replicas.

2.5 Replica State

Definition 2.8 (Replica State). The local state of replica r is a triple $\Sigma_r = (epoch_r, state_r, \mathcal{V}_r)$ where $epoch_r \in \mathbb{N}$ is the current epoch number (initially 0), $state_r \in \mathcal{S}$ is the CRDT state, and $\mathcal{V}_r$ is the dot version vector.

Definition 2.9 (Delta Message). A delta sent from replica s to replica r is a pair $(\delta, \hat{\mathcal{V}}_s)$. The sender's clock is included so the receiver can advance its own clock on merge. Delta exchange and GC operate on structurally disjoint dot sets: by Lemma 7.1, all dots in any live delta are strictly above the current epoch's threshold and therefore cannot interact with GC'd state.

3 Invariants

Invariant 3.1 (Clock Soundness). At all times: $dots(state_r) \subseteq \hat{\mathcal{V}}_r$.

Invariant 3.2 (Clock Monotonicity). $\hat{\mathcal{V}}_r$ is non-decreasing: no entry ever decreases.

Invariant 3.3 (GC Precondition). $gc(state_r, V)$ is only called when $V \leq \hat{\mathcal{V}}_r$.

Invariant 3.4 (Causal Floor). At the moment epoch N is committed, every replica $r \in \mathcal{M}_1$ satisfies $V_{stable}^N \leq \hat{\mathcal{V}}_r$: the GC threshold is the meet of their stored clocks at that instant, which are bounded above by their actual clocks by Invariant 3.2. Consequently, every live replica r at epoch N satisfies $V_{stable}^N \leq \hat{\mathcal{V}}_r$ at all times thereafter, since clocks are non-decreasing.

Invariant 3.5 (Initiator Currency). A replica may only initiate GC for epoch N if it is currently at epoch $N - 1$, i.e. $epoch_r = N - 1$ and no $epoch_entry[N']$ exists for any $N' > epoch_r$. It is *not* required that all other live replicas be at epoch $N - 1$: replicas that are behind will catch up via OBSERVEEPOCHCHANGE in a single step. Initiator currency is sufficient because the GC threshold V_{stable}^N is computed from all live replica clocks via COMPUTESTABLETIMESTAMP, regardless of whether those replicas have yet called OBSERVEEPOCHCHANGE for the previous epoch. By Invariant 3.4, every live replica already has $\hat{V}_r \geq V_{stable}^{N-1}$, so the meet of their clocks correctly reflects the true causal frontier.

Invariant 3.6 (Strict Threshold Progress). For any two consecutive committed epochs $N - 1$ and N :

$$V_{stable}^{N-1} < V_{stable}^N$$

i.e. $V_{stable}^N \geq V_{stable}^{N-1}$ and $V_{stable}^N \neq V_{stable}^{N-1}$. A GC epoch that would produce no advancement in the threshold is never committed.

Remark 3.1. Invariant 3.4 is the central safety invariant. Invariant 3.5 is its enforcement mechanism on the initiator side: a stale replica cannot install a V_{stable} computed from an incomplete view of the world. Invariant 3.6 ensures the epoch sequence makes genuine forward progress: every committed epoch strictly advances the GC threshold, preventing vacuous epochs and bounding the rate at which GC can be triggered.

4 Protocol Procedures

4.1 Normal Operation

Algorithm 1 Local Write at Replica r

```

1: procedure LOCALWRITE( $r$ , operation  $op$ )
2:    $dot_r.counter \leftarrow dot_r.counter + 1$ 
3:    $state_r \leftarrow apply(state_r, op, dot_r)$ 
4:   write( $clock[r]$ ,  $\hat{V}_r$ ) ▷ Periodically; not required on every write
5: end procedure

```

Algorithm 2 Merge Incoming Delta at Replica r

```

1: procedure MERGEDELTA( $r$ , message  $(\delta, V_{peer})$ )
2:    $state_r \leftarrow state_r \sqcup \delta$ 
3:    $\hat{V}_r \leftarrow \hat{V}_r \sqcup V_{peer}$ 
4: end procedure

```

Remark 4.1 (Delta Exchange is Unconditionally Safe). By Lemma 7.1, every dot in a delta generated by a live replica after epoch N satisfies $d \not\leq V_{stable}^N$. Delta exchange and GC therefore operate on structurally disjoint dot sets: the merge $state_r \sqcup \delta$ is correct regardless of the receiver's GC state, with no filtering, no epoch tags, and no guards required.

4.2 Computing the Stable Timestamp

Algorithm 3 Compute Causally Stable Timestamp

```

1: function COMPUTESTABLETIMESTAMP( $\mathcal{M}$ )
2:   return  $\bigcap_{r \in \mathcal{M}} readOrZero(clock[r])$  ▷ Missing clock treated as  $\mathbf{0}$ ; see Remark 4.2
3: end function

```

Remark 4.2 (Clock Conservatism and Missing Clocks). Each **readOrZero**(*clock*[*r*]) returns the clock value *r* last wrote, or **0** if the key is absent. Two sources of conservatism arise:

Stale clock. A present clock may be slightly behind *r*'s actual current clock, since *r* writes its clock periodically rather than on every operation. The resulting V_{stable} may therefore be strictly less than the true pointwise minimum of current clocks. This is safe: GC retains slightly more state than strictly necessary but never discards anything a live replica needs. The gap is bridged in the proofs by Clock Monotonicity (Invariant 3.2): the stored clock at commit time is a lower bound on the actual clock both at commit time and at all future times.

Missing clock. A replica that has written *member*[*r*] but not yet written *clock*[*r*] — as can happen when a new replica completes BOOTSTRAP but is observed by an initiator between its membership write and its clock write — contributes **0** to the meet. Since $\mathbf{0} \leq V_{stable}^{N-1}$ always holds, the resulting $V_{stable} \leq V_{stable}^{N-1}$, and the strict progress check in INITIATEGC aborts the epoch without committing it. No safety invariant is threatened: the epoch is simply deferred until the new replica has written its clock. This is a concrete instance of the general case covered by Invariant 3.6.

4.3 GC Initiation

Algorithm 4 GC Initiation at Replica r

```

1: procedure INITIATEGC( $r$ )
2:   OBSERVEEPOCHCHANGE( $r$ ) ▷ Ensure  $r$  is fully current before proceeding
3:    $N_{latest} \leftarrow \max\{N \mid epoch\_entry[N] \text{ is present}\}$ 
4:   if  $epoch_r \neq N_{latest}$  then
5:     return ▷  $r$  is behind; do not initiate GC
6:   end if
7:    $N \leftarrow epoch_r + 1$ 
8:   ▷ — Closed-world check —
9:    $\mathcal{M}_0 \leftarrow \text{listKeys}(\text{member}/)$ 
10:  if  $\neg \text{cas}(gc\_intent[N], \perp, N)$  then
11:    return ▷ Another replica claimed epoch  $N$ ; abort
12:  end if
13:   $\mathcal{M}_1 \leftarrow \text{listKeys}(\text{member}/)$ 
14:  if  $\mathcal{M}_1 \neq \mathcal{M}_0$  then
15:    delete( $gc\_intent[N]$ )
16:    return ▷ Membership changed during window; abort
17:  end if
18:  ▷ — Strict progress check —
19:   $V_{stable} \leftarrow \text{COMPUTESTABLETIMESTAMP}(\mathcal{M}_1)$ 
20:   $V_{prev} \leftarrow \text{read}(epoch\_entry[N - 1])$ 
21:  if  $V_{stable} \leq V_{prev}$  then
22:    delete( $gc\_intent[N]$ )
23:    return ▷ No advancement in threshold; GC would be a no-op
24:  end if
25:  ▷ — Commit epoch —
26:   $state_{gc} \leftarrow gc(state_r, V_{stable})$ 
27:  write( $epoch\_entry[N], V_{stable}$ )
28:  write( $bottom\_state[N], (state_{gc}, \hat{V}_r)$ )
29:  ▷ — Apply GC locally —
30:   $state_r \leftarrow state_{gc}$ 
31:   $epoch_r \leftarrow N$ 
32:  delete( $gc\_intent[N]$ ) ▷ Release barrier
33: end procedure

```

Remark 4.3 (Initiator Currency Enforcement). The call to OBSERVEEPOCHCHANGE at the start of INITIATEGC ensures that r incorporates all completed epochs before attempting to initiate a new one. The subsequent check $epoch_r = N_{latest}$ is a double guard: if OBSERVEEPOCHCHANGE found a new epoch and updated r , but another initiator concurrently committed yet another epoch between the call and the check, r detects this and aborts. A replica that is behind cannot install a V_{stable} computed from a stale membership view or stale clocks.

Remark 4.4 (Strict Progress Enforcement). The check $V_{stable} \leq V_{prev}$ gates the commit on genuine threshold advancement. $V_{stable} = V_{prev}$ arises when no replica has generated new operations since the last GC. The missing-clock scenario (Remark 4.2) produces $V_{stable} = \mathbf{0}$, which is a strict sub-case: $\mathbf{0} < V_{prev}$ always holds, so the check fires. In both cases the epoch is aborted and GC deferred without any safety violation.

Remark 4.5 (GC Trigger Policy). INITIATEGC is a pure mechanism: it defines what constitutes a valid GC epoch and enforces all safety conditions, but it does not specify *when* it should be

called. The trigger policy is intentionally left to the application layer. Recommended strategies include:

- **Minimum time interval.** Store a wall-clock timestamp alongside each $epoch_entry[N]$. An initiator checks that at least Δ_t time has elapsed since epoch $N - 1$ before proceeding.
- **Minimum dot accumulation.** Track the number of dots accumulated since the last GC. Trigger only when this count exceeds a threshold Δ_d .
- **Exponential backoff on abort.** When INITIATEGC aborts due to a lost CAS race or membership change, back off with exponentially increasing delay before retrying.
- **Designated initiator.** A single replica is statically or dynamically designated as the sole GC initiator, eliminating CAS contention entirely.

The strict progress check (Invariant 3.6) acts as a final backstop regardless of trigger policy.

Remark 4.6 (Closed-World Without a Wait). Reading $\mathcal{M}_0$ *before* the CAS and $\mathcal{M}_1$ *immediately after* eliminates the need for a bounded wait period. Let t_{cas} be the linearization point of the CAS. Any replica that wrote $member[r']$ before t_{cas} is visible in $\mathcal{M}_0$ (if its write linearized before the first **listKeys**) or in $\mathcal{M}_1$ (if it linearized between the first **listKeys** and the second). Either way, $\mathcal{M}_1 \neq \mathcal{M}_0$ and the initiator aborts. Any replica writing $member[r']$ after t_{cas} observes $gc_intent[N]$ present and waits (Section 4.4). The CAS itself is the synchronization barrier; no additional delay is required.

4.4 Joining a New Replica

Algorithm 5 New Replica Bootstrap

```

1: procedure BOOTSTRAP( $r_{new}$ )
2:   write( $member[r_{new}]$ ) ▷ Register freely; no coordination needed
3:    $N_{latest} \leftarrow \max\{N \mid bottom\_state[N] \text{ is present}\}$ 
4:   if read( $gc\_intent[N_{latest} + 1]$ )  $\neq \perp$  then
5:     wait until  $bottom\_state[N_{latest} + 1]$  is present ▷ GC in progress for next epoch; wait for completion
6:      $N_{latest} \leftarrow N_{latest} + 1$ 
7:   end if
8:    $(s_0, \hat{V}_{r_i}) \leftarrow \text{read}(bottom\_state[N_{latest}])$  ▷ Pair: GC'd state and initiator clock
9:    $V_{stable} \leftarrow \text{read}(epoch\_entry[N_{latest}])$ 
10:   $state_{r_{new}} \leftarrow s_0$ 
11:   $\hat{V}_{r_{new}} \leftarrow \hat{V}_{r_i} \sqcup V_{stable}$  ▷ Cover all dots in  $s_0$  via the initiator's clock
12:   $dot_{r_{new}} \leftarrow (r_{new}, 0)$ 
13:   $epoch_{r_{new}} \leftarrow N_{latest}$ 
14:  write( $clock[r_{new}], \hat{V}_{r_{new}}$ )
15: end procedure

```

Remark 4.7 (Checking Only the Next Epoch Intent). BOOTSTRAP checks for $gc_intent[N_{latest} + 1]$ specifically, not for any arbitrary intent. This is correct because Invariant 3.5 guarantees that any active GC initiator is at epoch N_{latest} and is preparing epoch $N_{latest} + 1$. An intent for epoch $N' \leq N_{latest}$ is impossible: such an epoch is already committed and its intent was deleted upon completion. An intent for epoch $N' > N_{latest} + 1$ is also impossible: no initiator can skip epochs.

Remark 4.8 (Why the New Replica Cannot See a Stale Intent). Since $gc_intent[N]$ is deleted by the initiator upon completing epoch N and object store operations are linearizable, a new replica that reads $gc_intent[N]$ as absent can be certain the key does not exist.

Remark 4.9 (Clock Write Timing). BOOTSTRAP writes $clock[r_{new}]$ as its final step. There is therefore a window between the $member[r_{new}]$ write and the $clock[r_{new}]$ write during which an initiator may observe r_{new} in $\mathcal{M}_1$ but find its clock absent. As described in Remark 4.2, COMPUTESTABLETIMESTAMP treats an absent clock as $\mathbf{0}$, driving V_{stable} to $\mathbf{0}$ and causing the strict progress check to abort the epoch. This is safe: no epoch is committed while r_{new} 's clock is unknown. A slow or crashed new replica therefore delays GC but never endangers correctness.

4.5 Observing an Epoch Change

Live replicas call this procedure upon detecting a new epoch or periodically to stay current.

Algorithm 6 Observe Epoch Change at Replica r

```

1: procedure OBSERVEEPOCHCHANGE( $r$ )
2:    $N_{latest} \leftarrow \max\{ N \mid epoch\_entry[N] \text{ is present} \}$ 
3:   if  $N_{latest} \leq epoch_r$  then
4:     return ▷ Already up to date
5:   end if
6:    $V_{stable} \leftarrow \text{read}(epoch\_entry[N_{latest}])$ 
7:    $state_r \leftarrow gc(state_r, V_{stable})$  ▷ GC local state to the new threshold
8:    $epoch_r \leftarrow N_{latest}$ 
9:   write( $clock[r]$ ,  $\hat{V}_r$ ) ▷ Publish current clock;  $V_{stable} \leq \hat{V}_r$  by Invariant 3.4
10: end procedure

```

Remark 4.10 (Correctness of ObserveEpochChange). OBSERVEEPOCHCHANGE does not modify $\hat{V}_r$ because the Causal Floor invariant already guarantees $V_{stable}^{N_{latest}} \leq \hat{V}_r$: this was established at epoch commit time (Theorem 11.1) and preserved by Clock Monotonicity (Invariant 3.2) ever since. The GC precondition (Invariant 3.3) is therefore satisfied. By Axiom 2.3, GC only removes dots, so Clock Soundness is preserved. The procedure does not merge $bottom_state[N_{latest}]$: operations the initiator had that r has not yet received carry dots strictly above V_{stable} (by Lemma 7.1) and will arrive via normal delta exchange.

Remark 4.11 (Single-Step Catch-Up). A live replica always performs exactly one state transition per epoch regardless of how many epochs it has missed: it jumps directly to N_{latest} by GC-ing to $V_{stable}^{N_{latest}}$. This is correct because Invariant 3.4 guarantees $\hat{V}_r \geq V_{stable}^N$ for every past epoch $N \leq N_{latest}$, so r 's state is already causally above all past thresholds.

4.6 Epoch Entry Cleanup

Algorithm 7 Epoch Entry Cleanup

```

1: procedure CLEANUP
2:    $N_{latest} \leftarrow \max\{ N \mid epoch\_entry[N] \text{ is present} \}$ 
3:   for each  $N < N_{latest}$  do
4:     delete( $epoch\_entry[N]$ )
5:     delete( $bottom\_state[N]$ )
6:   end for
7: end procedure

```

Remark 4.12 (Cleanup Safety). CLEANUP may be called by any replica at any time without coordination. It is safe because:

- **ObserveEpochChange** reads only $epoch_entry[N_{latest}]$ — never any older entry. Once a strictly newer epoch exists, no live replica will ever reference a stale epoch entry again.

- **Bootstrap** reads $bottom_state[N_{latest}]$. A new replica that reads $N_{latest} = N$ and then finds $bottom_state[N]$ deleted by a concurrent cleanup has simply raced with a new epoch being committed; it retries and finds $N_{latest} = N + 1$.
- **No membership tracking is needed.** Every live replica transitions directly to N_{latest} in a single step. Once $epoch_entry[N + 1]$ exists, epoch N 's entries are permanently unreferenced by any correct replica.

$gc_intent[N]$ is deleted by the initiator upon completing each epoch and requires no separate cleanup. $clock[r]$ entries are overwritten in place and are never deleted.

Part II

Correctness Proofs

5 Lemma 1: LocalWrite Preserves Clock Soundness

Lemma 5.1 (LocalWrite Preserves Clock Soundness). *If $dots(state_r) \subseteq \widehat{V}_r$ before LOCALWRITE, it holds after.*

Proof. LOCALWRITE increments $dot_r.counter$ to $n_r + 1$, producing fresh dot $d_{new} = (r, n_r + 1)$. The new state satisfies $dots(state'_r) = dots(state_r) \cup \{d_{new}\}$. Since $\widehat{V}'_r(r) = n_r + 1$, we have $d_{new} \leq \widehat{V}'_r$. For all pre-existing $d \in dots(state_r)$: $d \leq \widehat{V}_r \leq \widehat{V}'_r$ by Clock Monotonicity. $\square$ $\square$

6 Lemma 2: MergeDelta Preserves Clock Soundness

Lemma 6.1 (MergeDelta Preserves Clock Soundness). *If $dots(state_r) \subseteq \widehat{V}_r$ before MERGEDelta, then $dots(state'_r) \subseteq \widehat{V}'_r$ after.*

Proof. The updated clock is $\widehat{V}'_r = \widehat{V}_r \sqcup V_{peer}$. By Axiom 2.2, $dots(state'_r) \supseteq dots(state_r)$; every pre-existing dot satisfies $d \leq \widehat{V}_r \leq \widehat{V}'_r$. Every new dot $d \in dots(\delta)$ originates from the peer, whose Clock Soundness gives $d \leq V_{peer} \leq \widehat{V}'_r$. $\square$ $\square$

7 Lemma 3: No Future Dot Below Threshold

Lemma 7.1 (No Future Dot Below Threshold). *Let epoch N be committed with threshold V_{stable}^N . No live replica r can generate a dot d with $d \leq V_{stable}^N$ after epoch N has been committed.*

Proof. By Theorem 11.1, at the moment epoch N is committed, every live replica $r \in \mathcal{M}_1$ satisfies:

$$V_{stable}^N \leq \mathbf{readOrZero}(clock[r]) \leq \widehat{V}_r^{at\ commit}$$

The first inequality holds because V_{stable}^N is the meet of the stored clocks. The second holds by Clock Monotonicity (Invariant 3.2): the stored clock is a value r previously wrote and can only have been exceeded since then.

By Clock Monotonicity, $\widehat{V}_r^{later} \geq \widehat{V}_r^{at\ commit} \geq V_{stable}^N$ for all times after the commit. In particular, at the moment r generates a new dot $d = (r, n)$, it increments its own counter: $n = \widehat{V}_r(r) + 1 > \widehat{V}_r(r) \geq V_{stable}^N(r)$.

Therefore $d.counter > V_{stable}^N(d.replicaId)$, which means $d \not\leq V_{stable}^N$. $\square$ $\square$

Remark 7.1. Lemma 7.1 closes the proof gap identified in Theorem 12.1 (GC Safety): the claim that post-epoch dots are not dominated by V_{stable}^N is now a proved lemma rather than an inline assertion. It also justifies the delta safety argument in Section 4.1 and the no-filtering claim in MERGEDelta.

8 Theorem 1: Unique GC Initiator

Theorem 8.1 (Unique GC Initiator). *For any epoch number N , at most one replica successfully writes $gc_intent[N]$.*

Proof. $\text{cas}(gc_intent[N], \perp, N)$ is an atomic write-if-absent on the linearizable object store. By linearizability [Herlihy and Wing, 1990], among all concurrent CAS attempts on the same key, exactly one can observe the key absent and succeed; all others observe it present and fail. $\square$ $\square$

9 Theorem 2: Initiator Currency

Theorem 9.1 (Initiator Currency). *When INITIATEGC writes $epoch_entry[N]$, the initiator r satisfies $epoch_r = N - 1$, i.e. r is at the most recently committed epoch.*

Proof. INITIATEGC calls OBSERVEEPOCHCHANGE first, which sets $epoch_r = N_{latest}$. It then reads N_{latest} again and aborts if $epoch_r \neq N_{latest}$. Suppose another replica commits epoch N' between these two reads. Then N_{latest} increases, the check $epoch_r \neq N_{latest}$ fails, and the initiator aborts. If no such commit occurs, $epoch_r = N_{latest}$ and the initiator sets $N = epoch_r + 1 = N_{latest} + 1$. By Theorem 8.1, only one replica writes $gc_intent[N]$; the check ensures it is the replica currently at N_{latest} . $\square$ $\square$

10 Theorem 3: Closed-World Safety

Theorem 10.1 (Closed-World Safety). *When INITIATEGC proceeds past the membership check ($\mathcal{M}_1 = \mathcal{M}_0$), every replica that wrote $member[\cdot]$ before the CAS is in $\mathcal{M}_1$, and every replica that writes $member[\cdot]$ after the CAS bootstraps from $bottom_state[N]$.*

Proof. Let t_0 be the linearization point of the first **listKeys**, t_{cas} the linearization point of the CAS, and t_1 the linearization point of the second **listKeys**.

Case A: Replica r' writes $member[r']$ at time $t \in [t_0, t_1]$. By linearizability, this write is visible at t_1 , so $r' \in \mathcal{M}_1$. If $r' \notin \mathcal{M}_0$, then $\mathcal{M}_1 \neq \mathcal{M}_0$ and the initiator aborts.

Case B: Replica r' writes $member[r']$ at time $t > t_1$. Since $t > t_{cas}$ and $gc_intent[N]$ was written at t_{cas} , by linearizability r' 's subsequent read of $gc_intent[N_{latest} + 1]$ returns $N \neq \perp$. Per BOOTSTRAP, r' waits for $bottom_state[N]$ before proceeding.

In both cases, r' either causes an abort or bootstraps from the completed epoch. No new replica participates with causal state below V_{stable} . $\square$ $\square$

11 Theorem 4: Causal Floor Establishment

Theorem 11.1 (Causal Floor Establishment). *When INITIATEGC writes $epoch_entry[N] = V_{stable}$, every replica $r \in \mathcal{M}_1$ satisfies $V_{stable} \leq \hat{V}_r$.*

Proof. $V_{stable} = \bigcap_{r \in \mathcal{M}_1} \text{readOrZero}(clock[r])$. The strict progress check guarantees $V_{stable} > V_{stable}^{N-1} > \mathbf{0}$, so no stored clock is $\mathbf{0}$ (i.e. all clocks are present). Each stored clock satisfies $\text{readOrZero}(clock[r]) \leq \hat{V}_r^{current}$ by Clock Monotonicity (Invariant 3.2). Therefore:

$$V_{stable} \leq \text{readOrZero}(clock[r]) \leq \hat{V}_r^{current}$$

for every $r \in \mathcal{M}_1$, establishing Invariant 3.4. $\square$ $\square$

Corollary 11.2 (GC Precondition at Initiator). *The GC precondition (Invariant 3.3) is satisfied when the initiator applies $gc(state_r, V_{stable})$: since $r \in \mathcal{M}_1$, Theorem 11.1 gives $V_{stable} \leq \hat{V}_r$.*

12 Theorem 5: GC Safety

Theorem 12.1 (GC Safety). *Let epoch N be committed with threshold V_{stable}^N . For any state s_2 generated by a live replica after epoch N , all dots of s_2 satisfy $d \not\leq V_{stable}^N$, and:*

$$\text{gc}(s_1, V_{stable}^N) \sqcup s_2 \simeq s_1 \sqcup s_2$$

Proof. By Lemma 7.1, no live replica can generate a dot $d \leq V_{stable}^N$ after epoch N commits. Therefore $\text{dots}(s_2) \cap \{d \mid d \leq V_{stable}^N\} = \emptyset$.

Since no dot of s_2 is dominated by V_{stable}^N , the dots removed by $\text{gc}(s_1, V_{stable}^N)$ are disjoint from $\text{dots}(s_2)$ (by Axiom 2.3). The merge $\text{gc}(s_1, V_{stable}^N) \sqcup s_2$ therefore combines the same elements as $s_1 \sqcup s_2$, minus the GC-eligible dots from s_1 absent from s_2 . By the GC function definition, $\text{gc}(s_1, V_{stable}^N) \simeq s_1$, so $\text{gc}(s_1, V_{stable}^N) \sqcup s_2 \simeq s_1 \sqcup s_2$. $\square$ $\square$

13 Theorem 6: ObserveEpochChange Correctness

Theorem 13.1 (ObserveEpochChange Correctness). *After OBSERVEEPOCHCHANGE completes at replica r :*

- (i) $\text{epoch}_r = N_{latest}$.
- (ii) $V_{stable}^{N_{latest}} \leq \hat{V}_r$ (Invariant 3.4 holds).
- (iii) $\text{dots}(\text{state}_r) \subseteq \hat{V}_r$ (Clock Soundness preserved).
- (iv) $\text{state}_r \simeq \text{state}_r^{\text{pre}}$: observable value is unchanged.

Proof. (i) Set directly. $\checkmark$

(ii) $\hat{V}_r$ is not modified by OBSERVEEPOCHCHANGE. By Invariant 3.4, $V_{stable}^{N_{latest}} \leq \hat{V}_r$ was established at epoch commit time and maintained by Clock Monotonicity (Invariant 3.2) ever since. $\checkmark$

(iii) By Invariant 3.4, $V_{stable} \leq \hat{V}_r$, so the GC precondition (Invariant 3.3) holds. By Axiom 2.3, GC only removes dots: $\text{dots}(\text{gc}(\text{state}_r, V_{stable})) \subseteq \text{dots}(\text{state}_r) \subseteq \hat{V}_r$. No new dots are introduced. $\checkmark$

(iv) By the GC function definition, $\text{gc}(\text{state}_r, V_{stable}) \simeq \text{state}_r$. No merge with *bottom_state* is performed; operations not yet received from the initiator carry dots above V_{stable} (by Lemma 7.1) and will arrive via delta exchange. $\checkmark$ $\square$ $\square$

14 Theorem 7: Bootstrap Correctness

Theorem 14.1 (Bootstrap Correctness). *After BOOTSTRAP completes at r_{new} :*

- (i) $V_{stable}^{N_{latest}} \leq \hat{V}_{r_{new}}$ (Invariant 3.4).
- (ii) $\text{dots}(\text{state}_{r_{new}}) \subseteq \hat{V}_{r_{new}}$ (Clock Soundness).
- (iii) All future deltas from live replicas carry only dots above $V_{stable}^{N_{latest}}$ and are safe to merge.

Proof. (i) $\hat{V}_{r_{new}} \leftarrow \hat{V}_{r_i} \sqcup V_{stable} \geq V_{stable}$, where r_i is the epoch initiator. $\checkmark$

(ii) $\text{state}_{r_{new}} = s_0 = \text{gc}(\text{state}_{r_i}, V_{stable})$. By the initiator's Clock Soundness, $\text{dots}(\text{state}_{r_i}) \subseteq \hat{V}_{r_i}$. By Axiom 2.3, $\text{dots}(s_0) \subseteq \text{dots}(\text{state}_{r_i}) \subseteq \hat{V}_{r_i} \leq \hat{V}_{r_{new}}$. $\checkmark$

(iii) By Lemma 7.1, every live replica r satisfies that any dot it generates after epoch N_{latest} has $d \not\leq V_{stable}^{N_{latest}}$. By Theorem 12.1, merging such deltas into r_{new} 's GC'd state is safe. $\checkmark$ $\square$ $\square$

15 Theorem 8: Inductive Clock Soundness

Theorem 15.1 (Inductive Clock Soundness). *Clock Soundness (Invariant 3.1) is an invariant of the protocol: it holds at initialisation and is preserved by every procedure.*

Proof. **Initialisation.** $state_r = \perp, \hat{V}_r = \mathbf{0}: \emptyset \subseteq \mathbf{0}. \checkmark$

LocalWrite. Lemma 5.1. $\checkmark$

MergeDelta. Lemma 6.1. $\checkmark$

ObserveEpochChange. Theorem 13.1(iii). $\checkmark$

Bootstrap. Theorem 14.1(ii). $\checkmark$

InitiateGC (local GC). $gc(state_r, V_{stable})$ only removes dots (Axiom 2.3): $dots(gc(state_r, V_{stable})) \subseteq dots(state_r) \subseteq \hat{V}_r$. $\hat{V}_r$ is unchanged. $\checkmark$ $\square$ $\square$

16 Theorem 9: Strong Eventual Consistency

Theorem 16.1 (Strong Eventual Consistency). *Let replicas r and s both be at epoch N and suppose both have received the same set of operations $\mathcal{O}$ (directly or via $bottom_state[N]$). Then $state_r \simeq state_s$.*

Proof. (i) **Unique GC threshold.** By Theorem 8.1, epoch N has exactly one initiator, so V_{stable}^N is uniquely determined.

(ii) **Merge order independence.** By Axiom 2.1, $\sqcup$ is commutative, associative, and idempotent, so $\sqcup_{o \in \mathcal{O}} o$ is independent of merge order.

(iii) **Same pre-GC observable value.** Since both replicas received $\mathcal{O}$: $state_r^{pre} \simeq \sqcup_{o \in \mathcal{O}} o \simeq state_s^{pre}$.

(iv) **GC determinism.** By Axiom 2.3, $gc(\cdot, V_{stable}^N)$ is determined by the state and the CRDT's elig function. Applied to observably equivalent pre-GC states with the same unique threshold, and using Definition 2.2:

$$state_r \simeq gc(state_r^{pre}, V_{stable}^N) \simeq gc(state_s^{pre}, V_{stable}^N) \simeq state_s$$

$\square$

$\square$

17 Theorem 10: Strict Threshold Monotonicity

Theorem 17.1 (Strict Threshold Monotonicity). *For any two consecutive committed epochs $N-1$ and N :*

$$V_{stable}^{N-1} < V_{stable}^N$$

i.e. $V_{stable}^N \geq V_{stable}^{N-1}$ with strict inequality in at least one component.

Proof. **Well-foundedness** ($V_{stable}^N \geq V_{stable}^{N-1}$). By Invariant 3.4, every live replica $r \in \mathcal{M}_1$ satisfies $\hat{V}_r \geq V_{stable}^{N-1}$ at the time epoch N is initiated. Each stored clock satisfies $\mathbf{readOrZero}(clock[r]) \leq \hat{V}_r$ by Clock Monotonicity, and $\hat{V}_r \geq V_{stable}^{N-1}$. If all clocks are present, the meet $V_{stable}^N = \prod_r \mathbf{readOrZero}(clock[r]) \geq V_{stable}^{N-1}$. If any clock is absent, $\mathbf{readOrZero}$ returns $\mathbf{0} \leq V_{stable}^{N-1}$, so $V_{stable}^N \leq V_{stable}^{N-1}$ and the strict progress check aborts before any commit.

Strict advancement ($V_{stable}^N \neq V_{stable}^{N-1}$). The strict progress check in **INITIATEGC** aborts whenever $V_{stable}^N \leq V_{stable}^{N-1}$. An epoch N is therefore committed only when $V_{stable}^N > V_{stable}^{N-1}$, establishing both well-foundedness and strict advancement simultaneously for committed epochs.

$\square$

$\square$

Corollary 17.2 (No Vacuous Epochs). *Every committed epoch N strictly increases the set of GC-eligible dots: there exists at least one dot d with $d \leq V_{stable}^N$ and $d \not\leq V_{stable}^{N-1}$.*

18 Proof Dependency Table

Result	Depends On	
Lemma 5.1 (LocalWrite CS)	—	
Lemma 6.1 (MergeDelta CS)	Axiom 2.2, Inv. 3.2	
Lemma 7.1 (No Future Dot)	Thm. 11.1, Inv. 3.2	
Theorem 8.1 (Unique Initiator)	CAS linearizability	
Theorem 9.1 (Initiator Currency)	Thm. 8.1, OBSERVEEPOCHCHANGE	
Theorem 10.1 (Closed-World)	Thm. 8.1, linearizability, Inv. 3.5	
Theorem 11.1 (Causal Floor)	Inv. 3.2, Thm. 10.1, Inv. 3.6	
Corollary 11.2 (GC Precondition)	Thm. 11.1	
Theorem 12.1 (GC Safety)	Lem. 7.1, Axiom 2.3, Def. 2.2	
Theorem 13.1 (ObserveEpochChange)	Inv. 3.4, Inv. 3.2, Inv. 3.3, Axiom 2.3, Lem. 7.1, Def. 2.2	
Theorem 14.1 (Bootstrap)	Lem. 7.1, Thm. 12.1, Axiom 2.3	
Theorem 15.1 (Inductive CS)	Lemmas 5.1, 6.1, Thms. 13.1, 14.1, Axiom 2.3	
Theorem 17.1 (Strict Threshold Monotonicity)	Inv. 3.4, Inv. 3.2, Inv. 3.6	
Theorem 16.1 (SEC)	Thms. 8.1, 15.1, Axiom 2.1, Def. 2.2	

Table 1: Proof dependency and key references

References

- Almeida, P.S., Shoker, A., and Baquero, C. (2016). *Delta State Replicated Data Types*. Journal of Parallel and Distributed Computing, 111:162–173. arXiv:1603.01529.
- Bieniusa, A., Zawirski, M., Preguiça, N., Shapiro, M., Baquero, C., Balesgas, V., and Duarte, S. (2012). *An Optimized Conflict-Free Replicated Set*. INRIA Research Report 8083. arXiv:1210.3368.
- Herlihy, M. and Wing, J.M. (1990). *Linearizability: A Correctness Condition for Concurrent Objects*. ACM Transactions on Programming Languages and Systems, 12(3):463–492.
- Preguiça, N., Baquero, C., Almeida, P.S., Fonte, V., and Gonçalves, R. (2010). *Dotted Version Vectors: Logical Clocks for Optimistic Replication*. arXiv:1011.5808.
- Shapiro, M., Preguiça, N., Baquero, C., and Zawirski, M. (2011). *A Comprehensive Study of Convergent and Commutative Replicated Data Types*. INRIA Research Report 7506.
- Shapiro, M., Preguiça, N., Baquero, C., and Zawirski, M. (2011). *Conflict-Free Replicated Data Types*. In: Proceedings of SSS 2011, LNCS 6976, pp. 386–400. Springer.
- Zeller, P., Bieniusa, A., and Poetzsch-Heffter, A. (2014). *Formal Specification and Verification of CRDTs*. In: FORTE 2014, LNCS 8461, pp. 33–48. Springer.